Shared Requirements — All Framework Projects

Standing specification applied to every project prompt in this pack

Contents

Shared Requirements

Every project prompt in this pack assumes this document. Paste it into the target repo alongside the project-specific prompt, or paste both together as one brief.

The reference implementation of everything below is the **vLLM customer support assistant**. Where a requirement is ambiguous, that repo is the tie-breaker.

---

1. What every project must produce

A **demonstrable** system, not a notebook. The test is whether a prospective client can be shown a five-minute video and understand what it does, how it works, and why each component earns its cost.

That means three things must exist and work together:

1. A working product surface someone can actually use.
2. An architecture view that explains the system to a non-engineer.
3. A monitoring view that proves it is observable, secure and auditable.

---

2. Frontend: three tabs, mandatory

React + Next.js (App Router), TypeScript strict. Three top-level tabs.

Tab 1 — Product

The working demo. Whatever the project does — chat, voice, detection, recognition — this is where it is exercised live. It must be usable by someone who has never seen it, with no CLI and no configuration file editing.

Include a configuration surface where it makes sense for the domain (company name, policies, thresholds, tracked classes, enrolled identities). Configuration changes must visibly change behaviour, and the change must be inspectable before it is applied.

Tab 2 — Architecture Design

An interactive **3D force-directed graph** (`react-force-graph-3d`, which wraps `3d-force-graph`), built as an **expandable pipeline**, not a flat diagram.

This section is unusually prescriptive because the obvious implementations were each tried and each failed on camera. What follows is the design that survived.

The shape: a pipeline of expandable stage cards The graph opens showing **7–9 top-level stage cards** in the order a request actually travels, left to right. Each stage expands on click into its parts.

- **Pin one axis only.** Give each top-level stage a `flowOrder` and pin its x to `(flowOrder - (count-1)/2) * SPACING`. Leave y and z free.
  - Pinning two axes produces a flowchart drawn in WebGL. The first version of this pinned x and y; the client's words were *"it is kinda 2d"*.
  - Pinning none produces a sphere that is not architecture.
- **Child nodes are never pinned.** They cluster around their parent under the link force, which is what makes an expanded card read as *contained*.
- **The root is logical, not a node.** Parent every stage to a `ROOT_ID` that has no node of its own. Rendering a root produces a hub every stage hangs off — exactly the shape a pipeline is meant to replace, and it leaves the first stage visibly dangling.

Interaction: click to isolate, click again to return

- **Clicking a stage shows only that stage and its contents.** Everything else leaves the scene. Expanding in place leaves the thing being discussed competing with thirty other nodes — on a recording it becomes the *least* prominent element on screen.
- **Clicking the same node again restores the full network**, collapsed. The way back must be the thing you just clicked; a presenter should not have to hunt the toolbar mid-sentence. Provide `Esc` and a visible "Back to full network" button as well, because a view that hides everything needs a stated exit.
- **Nodes are draggable and stay where dropped** (set `fx/fy/fz` on drag end). The force layout is good at untangling and has no idea which arrangement makes the argument clearest.
- **The user owns the camera.** Auto-framing must stand down the moment someone drags. Implement it as: a pointer press that *moves* more than a few pixels claims the camera; a press that does not is still a click. Re-framing on every simulation settle is the single worst bug in this component — the graph springs back a beat after every drag and reads as "panning is broken".
- **Reset squares up before fitting.** `zoomToFit` only changes distance, so after a rotation it re-frames the pipeline while leaving it standing on end. Put the camera back on the +Z axis first, then fit.
- **Fit correctly.** If you tighten past `zoomToFit`, scale the camera position toward the **controls target**, not the world origin — the graph's bounding box is not centred on the origin, and scaling toward the origin swings the view direction and slides the graph into a corner.
- **Refit when the simulation stops**, not on a timer. A timed fit frames a layout that no longer exists a second later.

Node visuals: icons, and constant screen size

- **Every node carries a glyph** — a database cylinder, a shield, a chip, a person, a gateway. A client should recognise the database before reading a word. Shape survives video compression; 11px type does not.
- **Draw glyphs on a canvas; do not fetch logo files.** The Architecture tab has to render offline and in CI, and a missing logo mid-demo is worse than a simplified one. Brand SVGs are also trademarked artwork — a recognisable glyph in the product's own colour identifies the technology without redistributing someone's mark.
- **Nodes and labels must be constant size on screen** (`sizeAttenuation: false` on the sprite material). A pipeline spans thousands of world units while a node is ~10 across, so a perspective-

scaled node is under ten pixels at any framing that fits the whole diagram. The cost is losing size-as-depth-cue; the links and the flow axis already carry that.
- **Hang labels off the sprite's `center`, not a world-space offset.** A world offset shrinks toward nothing as the camera pulls back, and the label ends up sitting on top of the node it names.
- **Show that a card opens.** A collapsed parent needs a dashed ring or shell and a `+N inside` count. Without it, expandability is a secret.

Links

- **Roll every request-path edge up to the nearest *visible* ancestor.** Without this the collapsed view has no flow at all: every real edge runs between leaf nodes hidden inside collapsed cards, so the diagram shows containment and nothing else. Aggregating gives `Gateway → Context` because `authz → retriever` exists underneath it, and the edges resolve to finer granularity as cards open.
- **Copy link objects before handing them to the graph.** The library rewrites `source`/`target` from ids to node objects *in place*; passing your module-level constant means the second render is comparing objects against a set of ids and silently dropping every edge.
- **Give each relationship kind its own colour** — containment, request path, context being pulled in, data being written, observation, improvement feedback.
- **Set two rest lengths.** Containment edges short, so a card reads as one cluster. Flow edges *longer than the pinned spacing*, so the only way to satisfy them is to separate in y and z — that is what lifts the pipeline off the line into a ribbon with real depth, and what stops it settling flat.

The detail panel **HTML, never 3D text.** Prose rendered as WebGL is unreadable in a compressed video, and the cost figures are the part that has to be read. The scene carries structure; the panel carries meaning.

Each node's panel carries four fields, all mandatory:

Field	Answers
What it does	One line, plain language
Why it is used	Why this component exists at all — what breaks without it
Client benefit (cost)	What it saves, with a number wherever one has been measured
User benefit (quality)	What the end user actually feels

Plus a **"Say this"** line: the sentence the presenter speaks when the node is on screen. It is a script prompt that also reads as a summary to the viewer.

Where a component can be *shown* rather than described — the product's own UI, a sample detection, a rendered result — put a small worked example in its panel. Build it from the real components rather than embedding a screenshot: it must render with the backend off, it stays legible at any zoom, and it cannot drift out of date with the design system.

The guided tour An ordered walk through every significant node, with narration, arrow-key navigation and autoplay. **This is what gets recorded**, so it must:

- Open on the user's problem, not on the technology.
- Follow the pipeline in order and never double back.
- Expand every ancestor of a stop before selecting it, or the camera flies at a node still folded inside a collapsed parent and lands on nothing.
- Cover every component that carries the commercial argument.
- Include the honest stops — the technique that is *not* enabled, and why.

Non-negotiable **The Architecture tab must work with the backend switched off.** It is static data. A demo must never fail because a GPU is cold. Prove it with an end-to-end test that runs with no backend, rather than asserting it in a README.

Tab 3 — Monitoring

Four sections: **Quality**, **Performance**, **Security**, **Audit**.

- **Quality** — task-appropriate accuracy, and the rate at which the system correctly declines rather than guessing.
- **Performance** — latency percentiles (p50/p95/p99), throughput, hardware utilisation, and the framework's headline optimisation in action.
- **Security** — attacks detected and blocked, PII handling, authentication and authorisation failures, 4xx/5xx rates.
- **Audit** — an append-only, tamper-evident event log with compliance mapping.

Live data when available, seeded demo data otherwise, with a visible badge. The API response carries a `source` field (`"live"` or `"demo"`) and the UI renders the badge from it. Never present synthetic numbers as though they were live.

Frontend quality bar

The interface is being shown to buyers. Enforceable constraints, not aspirations:

- One neutral colour ramp plus **one** accent. Accent is reserved for focus, links and selection. Primary actions are near-black on light and near-white on dark — colour on a button should mean something specific, like destruction.
- No gradient hero sections, no glassmorphism, no purple/blue gradient accents, no emoji used as icons.
- Every colour comes from a design token. No ad-hoc hex values in components.
- WCAG AA contrast, visible `focus-visible` rings, `prefers-reduced-motion` honoured, full keyboard operation, correct ARIA on custom controls.
- Light and dark themes, following the OS by default with an explicit override.

---

3. Architecture: the component chain

Every project must include every one of these, adapted to its device class. If a component genuinely does not apply, say so explicitly in the node's detail panel and name the substitute — do not silently drop it.

One exception, and it is the one people get wrong: RAG. Retrieval-augmented generation belongs in a system that answers questions from a body of documents. It does **not** belong in a face-recognition system, an object tracker or a drone-imagery pipeline. Those have a *context assembly* step — an enrolment index, a frame buffer, a tile with its geo-reference — and that is a different thing wearing the same slot in the diagram.

Do not put a "RAG retrieval" node in a vision project. There is no corpus, there is no query embedding, and a client who works in the domain will spot it immediately and start discounting everything else on the page. Name the real component instead:

Project	The context-assembly step is actually
Voice agent, document analysis, offline assistant	**RAG** — genuine retrieval over a document corpus
Face recognition	**Enrolment index lookup** — cosine similarity over stored templates
Object tracking	**Frame pipeline + track state** — the previous frames and open tracks
Drone imagery	**Tiling + geo-reference** — the tile the model sees and where it is in the world

The same discipline as section 4's KV-cache rule: uniformity across the pack is worth nothing next to being right about the domain.

```
User action (Product tab)
  → Client application
  → Edge: ingress, TLS, rate limiting
  → Authentication (who are you)
```

```
→ Authorization (what may you do)
→ Request analysis / intent
→ Context assembly — RAG *only if* there is a document corpus to retrieve
     from; otherwise the real equivalent (enrolment index, track state,
     tile + geo-reference). See the note below.
→ Skills / tools / business logic
→ INFERENCE ENGINE  ← the framework this project is about
     + its headline optimisation (see section 4)
     + its compression strategy (quantization, pruning, distillation)
→ Post-processing, guardrails, confidence gating
→ Stateless structured logging (so instances scale horizontally)
→ Audit log: append-only, hash-chained, HIPAA / SOC 2 / GDPR mapped
→ Monitoring: latency, throughput, 4xx/5xx — Prometheus + Grafana
→ Staging and production environments
→ Orchestration and scaling
→ IMPROVEMENT LOOP  ← feedback capture → preference data → training → gate
     (section 5a — the only edges that flow back upstream)
```

Stateless is a requirement, not a preference. No inference or application node may hold session state in memory. State belongs in the database or the client. This is what allows horizontal scaling, and it is worth calling out on the node.

---

4. Optimisation: substitute honestly

This is the rule most likely to be got wrong, so it is stated bluntly.

> **KV cache, continuous batching and prefix caching exist only where there is autoregressive token generation.**

Project	KV cache applies?	Headline optimisation to demonstrate instead
TensorRT-LLM voice agent	**Yes**	Paged KV, in-flight batching, fused kernels, FP8
SGLang document analysis	**Yes**	RadixAttention prefix reuse, structured decoding
llama.cpp offline assistant	**Yes**	GGUF quantization, `mmap` load, Metal/NNAPI offload
ExecuTorch face recognition	**No**	INT8 PTQ, XNNPACK/Core ML delegation, operator fusion
ONNX Runtime object tracking	**No**	INT8 quantization, graph fusion, execution providers
LiteRT drone imagery	**No**	INT8 full-integer quant, NPU/EdgeTPU delegation, pruning

For the three vision projects, **do not fabricate a KV cache**. A detection model runs one forward pass per frame — there is no autoregressive state to cache. The node in the architecture graph should be labelled with the real optimisation and its detail panel should say plainly why KV caching does not apply here. A client who knows the domain will notice, and being right is worth more than being uniform.

Likewise for orchestration:

Device class	“Staging production” means
GPU server	Kubernetes, staged rollout, HPA on the right signal
Mobile	Model registry + staged OTA rollout rings, phased release
IoT / edge	Fleet management, canary devices, signed model artifacts, rollback

A phone does not run Kubernetes. The requirement is a **controlled path from staging to production with a rollback**, not the specific tool.

---

5. Compression is mandatory, and it must be gated

Every project compresses its model, and every project proves the compression did not break it.

- State the technique, the target hardware, and the memory before and after.
- Explain **what the freed memory buys** — that is the client-facing point. On a GPU server, smaller weights become KV cache, which becomes concurrent users. On a phone, it becomes the difference between running and not running.
- Ship a **quality gate** that fails the build when accuracy drops past a threshold. Compare against the uncompressed baseline. Make the scoring deterministic — a gate that blocks releases must give the same answer twice.
- Include a small task-specific behavioural suite alongside any academic benchmark. General benchmarks catch capability damage; only a domain suite catches the model becoming useless at the actual job.

5a. RLHF: the improvement loop is mandatory

Everything above describes a system that stays the same. Ship the part that compounds: **reinforcement learning from human feedback** — human judgements about outputs become the training data for the next version of the model.

"RLHF" here means the modern, practical form of it: collect preferences, train with DPO on a LoRA adapter, gate the result. Not a reward model and a PPO loop — that is far more machinery for the same signal, and on a few thousand domain preferences the reward model would be the weakest part of the pipeline. Where a project has no generative model to fine-tune (the vision projects), the same loop runs with the training step swapped for retraining or threshold policy; say which and do not pretend otherwise.

This is also the strongest commercial argument in the deck. It is the difference between "the accuracy you buy is what you get" and "the accuracy you buy is a floor" — and unlike a hosted API, the judgements your client's team makes stay theirs.

Collection is automatic

Build these into the Product tab, not into a separate admin tool:

Signal	What it is	Why it is worth collecting
Rating	Thumbs up / down on one output	Cheap and plentiful; weak on its own, because people disagree about what "good" means in the abstract
Preference	"This output is better than that one" for the same input	Far stronger. A comparison sidesteps the calibration problem entirely, and it is the native input format for DPO and for reward-model training
Correction / comment	Free text, or the corrected label	Not directly trainable, but the only signal that says *why*, and the one that tells you which failure to go fix

Rules that are easy to get wrong and expensive to fix later:

- **Ask at the moment of judgement.** Under the output, while the person still has an opinion. Ask an hour later and you collect recall, not judgement.

- **Use ±1, not a 1–5 scale.** Five-point scales collapse to their extremes and the middle carries no usable signal, at the cost of a harder decision.
- **Comparisons must share their context.** Both candidates get the identical retrieved documents / identical input frame, so the judgement is about the model rather than about which side got luckier inputs.
- **Hide which variant is which until after the choice.** Knowing that A is "the current settings" is enough to bias the answer toward it, and a preference set with that bias baked in teaches the model to prefer what it already does.
- **Redact personal data on write.** This table is an export surface, and an export is the easiest way for personal data to leave the building — after which it is memorised by a model and cannot meaningfully be deleted.
- **Stamp the config/model version on every judgement.** A preference collected against an older prompt or an older model describes a system that no longer exists. Training on a mixture without knowing which is which drags a model toward its own past.
- **Store the triple denormalised** — `(input, chosen, rejected)`. The exported example must be exactly what the annotator saw, and it must survive the source record being deleted under retention.
- **Mark what an export consumed**, so the export is resumable and the same judgement is not trained on twice and quietly over-weighted.

For the vision projects the same loop applies with the modality swapped: the judgement is *"this box* / *this identity* / *this classification is wrong"*, and a correction is a re-labelled frame. The pipeline shape does not change.

Training is *not* automatic, and say so

The loop stops at an export. A fine-tune that fires whenever enough judgements accumulate is a way to ship a regression that nobody read the data for.

- Export as JSON Lines in the format the trainer reads with no conversion step — a conversion step is exactly where a schema drifts out of sync with the trainer.
- Prefer **DPO over reward-model + PPO** for generative projects: it optimises directly on preference pairs with far less machinery to get wrong.
- Prefer a **LoRA adapter over a full fine-tune**: a few hundred MB, hours on one GPU, loadable at runtime, and rollback is unloading a file. The low-rank constraint is also a real guard against a narrow preference set making the model worse at everything else.
- **Know which half of the system LoRA lives in.** On a server runtime that supports adapters (vLLM, SGLang, TensorRT-LLM with the plugin compiled in) it is a *serving* technique too: one base model plus a small adapter per tenant, per document type or per vertical, selected at request time — which turns "one deployment per customer" into "one GPU, many specialisms" and is usually the strongest cost argument available. On an exported edge runtime (ExecuTorch, ONNX Runtime, LiteRT) it is a *training-only* technique: adapt, **merge, then export and quantize**. There is no adapter swapping on a frozen graph, and an architecture diagram that shows one in the serving path is wrong. Each project prompt states which case it is in.
- **Refuse to train below a floor** (a few hundred pairs). Below that you produce a model with strong opinions about a handful of inputs and worse behaviour everywhere else.
- Write a **training manifest** next to the artifact: base model, pair count, source file, hyperparameters. Six months later the only question that matters about a model in production is what it was trained on.

The Monitoring tab gets a fifth section

Add **Improvement** alongside Quality / Performance / Security / Audit: approval rate, judgements collected, head-to-head win rate of the challenger, and how many judgements are waiting for the next training run.

This is the one panel that is never seeded with demo data. Every other section falls back to synthetic numbers when the metrics backend is absent and says so with a badge. Feedback is a claim about what real people judged, and a plausible approval rate nobody gave is the one number on the dashboard that cannot be corrected by waiting for real traffic. An empty loop renders as empty.

5b. Nothing is promoted without beating what it replaces

Any artifact that changes the weights — quantization, pruning, or a fine-tune from collected feedback — faces the same gate, on **two axes that genuinely come apart**. A fine-tune can answer better *and* stream slower; a change that speeds generation up can cost accuracy. A gate that asks only one question ships the other regression.

Dimension	Question	Tool (generative)	Tool (vision / edge)
Quality	Is it still right?	`lm_eval` — instruction following, reasoning, plus a held-out domain suite	Task metrics on a held-out set: mAP, IoU, TAR@FAR, top-1
Performance	Is it still fast and efficient?	`GuideLLM` against a running server — p95 TTFT, inter-token latency, tokens/sec	On-device latency p95, memory peak, FPS, energy per inference

Rules:

- **Benchmark through the real serving path**, not an in-process engine. The number that matters is what a user experiences, including the scheduler, batching and transport. An in-process benchmark measures a configuration nobody runs.
- **Hold a realistic arrival rate**, not "as fast as possible". Saturating the server measures peak throughput, which is not the operating point.
- **Relative bounds where a baseline exists** — "20% slower" means the same thing on any hardware, "+40ms" does not. **Absolute ceilings** where a number is unshippable regardless of history: a model that was already slow does not license the next one to be slower.
- **Absolute floors for behaviour.** A model that ignores the supplied policy, or that fails a liveness check, is unshippable whatever it scored on a benchmark.
- **Deterministic scoring.** A gate that blocks releases must give the same answer twice — so assertion-based, greedy decoding, no LLM judge adding its own variance to the signal being measured.
- **Either axis failing exits non-zero** and the artifact is not promoted.

Put both benchmarks in the architecture graph as their own nodes under the promotion gate. "We measure quality and speed before shipping" is a claim; two named tools with stated thresholds is evidence.

---

6. Security, audit and compliance

Security - Authentication and authorisation as separate, visible components. - Input validation and rate limiting at the edge. - Adversarial input detection appropriate to the modality — prompt injection for text and vision-language, and for the vision projects, at minimum an input sanity/liveness check where the domain calls for one. - Content received from a retrieval system or an uploaded document is **data, never instructions**. Treat and test it as an attack surface. - The inference endpoint is never internet-reachable. Only the application tier may reach it, enforced at the network layer. - No long-lived credentials. Workload identity or equivalent.

Audit - Append-only event log: actor, action, resource, outcome, timestamp. - Hash-chained so tampering is detectable. This is the property compliance buyers ask about, and it is cheap to implement. - Retention policy stated explicitly. - Personal data redacted **on the write path**, not filtered at read time. A filter someone can forget to apply is not a control.

Compliance mapping — for each audit event type, state which control it serves:

Regime	What must be demonstrable
GDPR	Lawful basis, data minimisation, erasure (Art. 17), special-category handling (Art. 9) where biometrics are involved

Regime	What must be demonstrable
HIPAA	Access logging, minimum necessary, encryption at rest and in transit
SOC 2	Change management, access control, monitoring, incident evidence

Do not claim certification. Claim *"designed to support"* and show the evidence the control would need. Overclaiming here is a liability, not a selling point.

---

7. Monitoring: measure the right signal

Prometheus for metrics, Grafana for dashboards, structured JSON logs with a request ID echoed to the caller.

Two dashboards, because they answer different questions and can disagree:

1. **System** — latency, throughput, saturation, errors, hardware utilisation.
2. **Product** — is it actually useful? Task success rate, decline rate, confidence distribution, cost per successful outcome.

A system can be green on every infrastructure metric and useless. Only the product dashboard reveals that.

Autoscale on the signal that actually saturates. For GPU inference this is usually queue depth, not CPU — a saturated inference process is blocked on the accelerator while its CPU sits at 30%, so a CPU-target autoscaler never fires. Work out the real saturation signal for the project's hardware and scale on that.

Alerts must be actionable. Every rule carries a runbook line saying what to check.

---

8. Engineering standards

- **Tests**: unit tests for the logic that carries the product's promises. If a component's correctness is the selling point, it has a test.
- **Types**: strict mode, no escape hatches without a comment explaining why.
- **CI**: lint, types, tests, and schema validation of any generated manifests.
- **Docker Compose** for a one-command local stack; the same images run in production.
- **Infrastructure as code** for anything cloud-side.
- **README** that a stranger can follow to a working demo.

Comments explain *why*, never *what*. The reasoning behind a non-obvious decision — why this layout, why this order, why this instance type — is the most valuable thing in the repo and the thing you will point at on camera. Dense, uncommented code looks efficient and reads as unconsidered.

8a. The node data model, and the README section that closes the argument

One typed record per node

The architecture graph is content, and content rots silently: a renamed id breaks a tour stop, a missing field renders an empty panel, and neither shows up until someone is recording. Keep it in **one typed file** with **tests**.

```
interface ArchNode {
  id: string;
  label: string;
  tier: TierId;          // which stage it belongs to — drives colour and filters
  parent?: string;       // hierarchy; ROOT_ID is logical and has no node
  flowOrder?: number;    // top-level stages only — the pinned axis
  sub?: string;          // one-line technical detail shown under the label
```

```
  icon?: IconKey;        // canvas-drawn glyph
  size?: number;

  what: string;          // the four mandatory rationale fields
  whyUsed: string;
  clientBenefit: string;
  userBenefit: string;

  metric?: { value: string; caption: string; estimated?: boolean };
  demoNote: string;      // the "Say this" line
}
```

Tests that are cheap and fail loudly — write all of them:

- Every node id is unique; every link connects two real nodes.
- Every node reaches the root; no parent cycles.
- Every node has all four rationale fields, non-trivially long.
- Every tour stop points at a real node, and the tour never goes backwards through the pipeline.
- Every icon key a node asks for exists — a typo indexes `undefined` and throws inside the render loop, taking the whole canvas down rather than one glyph.
- Every top-level stage has a glyph and a distinct `flowOrder`.
- The mandated component chain from section 3 is complete, asserted **by concept with a regex** rather than by id, so a rename cannot silently drop one.
- Anything not actually enabled is described as not enabled.

Be honest about what is not on

Include at least one node for a technique that is **built but deliberately disabled**, with the reason. In this reference implementation that is 2:4 pruning: the recipe ships, it is off by default, because one-shot sparsity costs instruction-following and that is the wrong thing for a support assistant to lose.

A client who checks one overstated claim discounts every other number in the deck. A stated limitation does the opposite — it makes the rest credible, and it is the single highest-leverage paragraph in the whole presentation.

The README must end with the tradeoff triangle

Every choice in these projects is a point on the same triangle: **cost, performance, accuracy**. Close the README with an image of it and a table where each row is one real decision, what it bought, and what it paid with. For example:

Decision	Bought	Paid with
INT4 instead of fp16	Weights that fit a commodity GPU	A small accuracy risk, which is why compression is gated rather than trusted
Pruning left **off**	Nothing — it ships disabled	A higher memory floor, because sparsity costs instruction-following
Escalating instead of guessing	Answers a user can trust	Deflection rate itself — every handoff is a task not resolved
Self-hosting rather than an API	Fixed cost that does not scale with success	Operational work: a model to run, update and roll out

Then say why the Monitoring tab exists: the triangle is not a one-time decision. Cost, latency and quality are measured continuously and side by side, so a change that improves one at the expense of another is visible rather than discovered later by a customer.

Where the framework supports it, add a second closing section on **portability** — which models, which accelerators, which deployment targets the same code reaches without a rewrite. Committing to a hosted

API means committing to one vendor's model, pricing and deprecation schedule simultaneously; keeping those as three separate reversible decisions is a commercial argument, not a technical one.

---

9. Deliverables checklist

Every project is done when all of these are true:

- ☐ Three tabs present and working
- ☐ 3D architecture graph: expandable pipeline, click-to-isolate, draggable nodes, icon glyphs, constant-size labels, and a guided tour
- ☐ Camera never fights the user — auto-framing stands down on drag
- ☐ Every component from section 3 present as a node with all four rationale fields plus a "Say this" line
- ☐ Architecture tab works with the backend off, proven by a test
- ☐ Feedback capture in the Product tab: rating, comparison, correction
- ☐ Preference export in the trainer's own format, resumable
- ☐ Monitoring has a fifth Improvement section, never seeded
- ☐ Release gate measures quality *and* performance, and can fail on either
- ☐ README carries the cost/performance/accuracy tradeoff table
- ☐ Model compressed, with a quality gate that can fail
- ☐ Monitoring shows live-or-demo data with an honest badge
- ☐ Audit log append-only and hash-chained
- ☐ Prometheus metrics exposed; Grafana dashboards committed
- ☐ Staging production path with rollback
- ☐ One-command local run
- ☐ CI green: lint, types, tests
- ☐ README a stranger can follow
- ☐ Measured performance numbers recorded — not estimated